

Introduction to Neural Networks

Biology, the McCulloch–Pitts Neuron, and Linear Separability

Introduction to Deep Learning ·

Outline

The Biological Neuron

From Neuron to Computing Unit

The McCulloch–Pitts Unit

Computing with Threshold Logic

Geometry: Linear Separability

The Biological Neuron

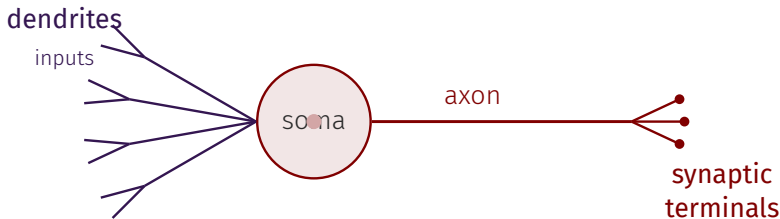
The brain as an inspiration

- The brain: roughly 10^{11} neurons, each connected to up to 10^4 others — massively **parallel**, not sequential
- Slow components ($\sim$ ms switching) yet faster than any computer at perception and motor control
- Rojas: artificial neural networks are best understood as **networks of primitive functions** — simple units, wired together

Rojas (1996), §1.3

The plan: abstract the essential behaviour of a neuron into a simple computing unit, then study what networks of such units can do.

Anatomy of a neuron



- **Dendrites** collect incoming signals; the **soma** integrates them
- If the integrated signal crosses a threshold, the neuron **fires** an impulse down the **axon**
- **Synapses** pass the signal on — some excitatory, some inhibitory Rojas (1996), §1.2

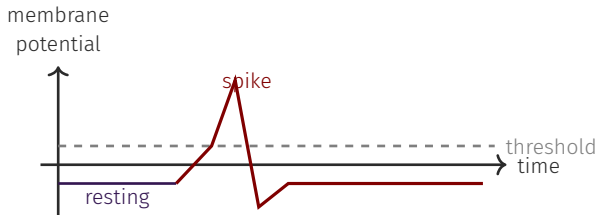
Firing: an all-or-nothing threshold

- A neuron integrates its inputs; when the membrane potential exceeds a **threshold**, it emits an **action potential** — otherwise it stays silent
 - The spike is **all-or-nothing**: its shape does not encode magnitude — only *whether* and *how often* the neuron fires
 - This thresholding behaviour is the single feature we will keep in our first model
- cf. NPTEL Comp. Neuroscience, simple neuron models

BIO membrane potential crosses a threshold → spike

MODEL weighted sum crosses a threshold → output 1

How the signal travels: the action potential



- Below threshold, nothing happens; **cross it** and the neuron fires a sharp, stereotyped pulse
- The pulse travels down the axon **without losing strength** — a digital signal on a noisy wire
- After firing, a brief **refractory** dip: the neuron resets before it can fire again

Rojas (1996), §1.2.2

At the synapse: adding and subtracting

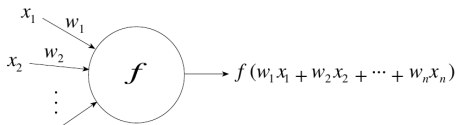
- Neurons meet at **synapses**; a spike releases chemical messengers that nudge the next neuron
- **Excitatory** synapses push the membrane *toward* threshold; **inhibitory** synapses push it *away*
- The soma continuously **sums** these pushes — thousands of them — and fires only if the net effect clears the threshold
- A single strong inhibitory input can **veto** many excitatory ones Rojas (1996), §1.2.3

BIO excitatory +, inhibitory −, integrated
at the soma

MODEL positive and negative **weights**,
summed in the unit

From Neuron to Computing Unit

The abstract neuron



Rojas (1996), Fig. 1.14.

- Each input channel i carries a value x_i , scaled by a **weight** w_i
- The body integrates the inputs (usually a sum) and applies a **primitive function** f
- Output: $f(w_1x_1 + \dots + w_nx_n)$

BIO dendrites $\times$ synaptic strengths,
summed at the soma

MODEL inputs $x_i \times$ weights w_i , summed in
the unit

Networks of primitive functions

- An artificial neural network is nothing but a network of primitive functions

Rojas (1996), §1.3.1

- Models differ only in three choices:

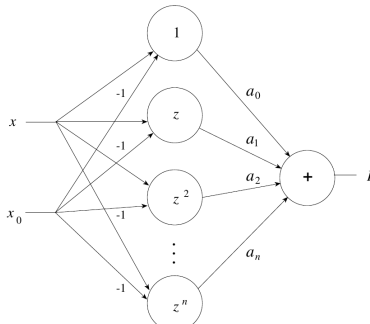
the primitive
functions used

the interconnection
pattern

the timing of
transmission

- Today's primitive function: the simplest possible — a threshold

Networks can approximate functions



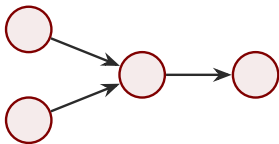
Rojas (1996), Fig. 1.16: a Taylor network.

- Choose the primitive functions as powers $z, z^2, \dots, z^n$ and the network computes a **Taylor series**
- Choose sines and it computes a **Fourier series** instead
- The lesson: a network is a **function approximator** — its power comes from *which* primitives you compose and how
- The weights $a_0, \dots, a_n$ can even be **learned** from data

Rojas (1996), §1.3.2

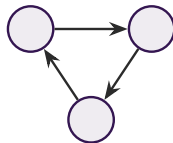
The McCulloch–Pitts Unit

Feed-forward vs. recurrent



feed-forward

flows one way, no cycles

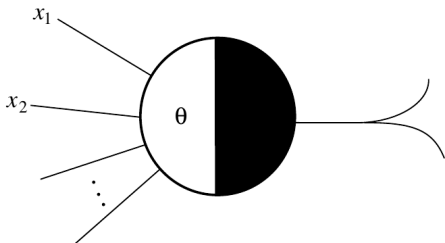


recurrent

has cycles: memory

- **Feed-forward**: a directed acyclic graph — output is a fixed function of the input
- **Recurrent**: cycles let signals loop back, giving the network **state** and memory
- Chapters 2–7 (and this course) focus on feed-forward networks Rojas (1996), §2.1.1

The McCulloch–Pitts unit (1943)



Rojas (1996), Fig. 2.6.

- Inputs and output are **binary** (0 or 1); edges are unweighted
- Edges are **excitatory** or **inhibitory** (small circle)
- Each unit has a threshold θ ; drawn as a circle with a black half Minsky's convention

BIO excitatory / inhibitory synapses

MODEL excitatory / inhibitory edges into a threshold unit

The firing rule

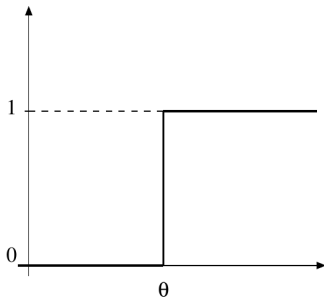
A unit receives $x_1, \dots, x_n$ on n excitatory edges and $y_1, \dots, y_m$ on m inhibitory edges (Rojas, 1996, §2.1):

- If $m \geq 1$ and **any** inhibitory signal $y_j = 1$: the unit is **inhibited**, output = 0
- Otherwise compute $x = x_1 + \dots + x_n$ and compare with θ : output = 1 if $x \geq \theta$, else 0
- With no inhibition, the unit acts as a **threshold gate**

BIO fire iff input exceeds threshold; inhibition can veto

MODEL output 1 iff $x \geq \theta$; any active inhibitor forces 0

The activation is a step function



Rojas (1996), Fig. 2.7.

- The output jumps **discontinuously** from 0 to 1 at $x = \theta$
- This is the **step** (Heaviside) activation function
- Its hard threshold mirrors the neuron's all-or-nothing spike

BIO action potential: all-or-nothing

MODEL step function: output flips at θ

Computing with Threshold Logic

Boolean functions from threshold gates

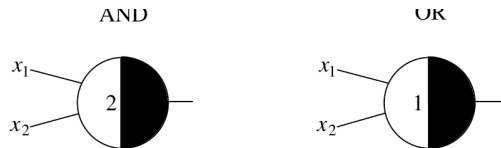


Fig. 2.8. Implementation of AND and OR gates

Rojas (1996), Fig. 2.8.

Two inputs, associate 1 = *true*, 0 = *false*:

- **AND**: threshold $\theta = 2$ — fires only if *both* inputs are 1
- **OR**: threshold $\theta = 1$ — fires if *either* input is 1

A single unit already computes fundamental logic gates.

BIO a neuron as a coincidence detector

MODEL a threshold gate computing AND / OR

Threshold logic is expressive

- A single unit can compute AND or OR of n arguments — just set $\theta = n$ (AND) or $\theta = 1$ (OR)
 - Negation needs an inhibitory edge: a unit with $\theta = 0$ and one inhibitory input computes NOT
 - With AND, OR, and NOT available, threshold-logic networks can compute any Boolean function
- Rojas (1996), §2.2

But how expressive is a *single* unit? The answer is geometric.

Any Boolean function, by construction

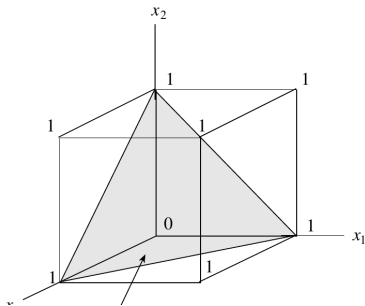
A logical function is a **table**: for each input vector, a 0 or a 1. To build a network for it (Rojas, 1996, §2.2.3):

1. For every input row that should output 1, use one unit as a **decoder** — it fires *only* for that exact vector
2. A decoder for $(1, 0, 1)$: excitatory edges from the 1-bits, inhibitory from the 0-bits, threshold = number of 1-bits
3. Feed all decoders into a single **OR** unit

Two layers of threshold units suffice for *any* Boolean function — expressive power is not the problem.

Geometry: Linear Separability

A unit splits input space into half-spaces



Rojas (1996), Fig. 2.12: the OR function.

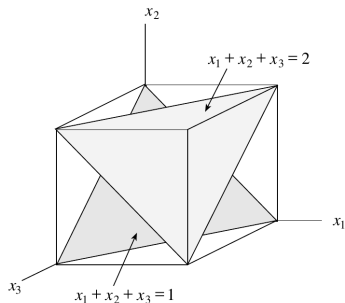
For input (x_1, x_2, x_3) and threshold θ , the unit tests $x_1 + x_2 + x_3 \geq \theta$:

- The equality $x_1 + x_2 + x_3 = \theta$ is a **separating plane**
- One side fires (1), the other does not (0)
- OR: $\theta = 1$; only the all-zero vertex lies below the plane

BIO a neuron responds to a region of stimulus space

MODEL a unit responds to a *half-space* of inputs

Parallel planes, different thresholds

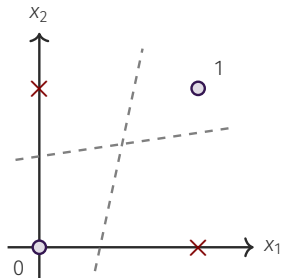


• Separating planes of the OR and majority

Rojas (1996), Fig. 2.13: OR and majority.

- **OR**: plane $x_1 + x_2 + x_3 = 1$
- **Majority**: plane $x_1 + x_2 + x_3 = 2$ — fires when at least two inputs are 1
- For McCulloch–Pitts units (unit weights) the separating planes are always **parallel**
- **Non-parallel** planes require **weighted** edges — the perceptron (Class 3)

The one it cannot do: XOR



$\times$: XOR = 1; $\circ$: XOR = 0.

- XOR outputs 1 when the inputs **differ**, 0 when they agree
- The two $\times$ points sit on **opposite corners** — so do the two $\circ$ points
- **No single straight line** can put both $\times$ on one side and both $\circ$ on the other
- XOR is **not linearly separable** — a wall a single unit cannot climb

Linear separability — and its limit

- A single threshold unit can compute exactly the **linearly separable** functions: those whose 1- and 0-inputs can be split by a hyperplane
- AND, OR, majority — all linearly separable
- XOR is not — and most functions of many variables are not, either

A single unit is a linear classifier. Escaping linear separability needs **weights** (Class 3) and, ultimately, **layers** (Classes 5–7).

Takeaways

1. A neuron integrates weighted inputs and fires when a **threshold** is crossed — all-or-nothing
2. The **McCulloch–Pitts unit** keeps only this: binary inputs, a threshold, excitatory/inhibitory edges
3. A single unit computes AND, OR, majority — but only **linearly separable** functions
4. Geometrically it is a **separating hyperplane**; XOR already escapes it

Class 3: add **weights** and a **learning rule** — the perceptron.

References

- R. Rojas. *Neural Networks: A Systematic Introduction*. Springer, 1996. Chapters 1–2. Figures 1.14, 2.6, 2.7, 2.8, 2.12, 2.13 reproduced with attribution.
- W. S. McCulloch & W. Pitts (1943). “A logical calculus of the ideas immanent in nervous activity.” *Bulletin of Mathematical Biophysics*, 5, 115–133.
- NPTEL Computational Neuroscience notes (simplified neuron models; perceptrons), used for supporting intuition.

Questions?